

```

from http import HTTPStatus

from django.contrib.auth import get_user_model
from django.test import Client, TestCase
from django.urls import reverse

from news.forms import BAD_WORDS, WARNING
from news.models import Comment, News

User = get_user_model()

class TestCommentCreation(TestCase):
    COMMENT_TEXT = 'Текст комментария'

    @classmethod
    def setUpTestData(cls):
        cls.news = News.objects.create(title='Заголовок', text='Текст')
        # Адрес страницы с новостью.
        cls.url = reverse('news:detail', args=(cls.news.id,))
        # Создаём пользователя и клиент, логинимся в клиенте.
        cls.user = User.objects.create(username='Мимо Крокодил')
        cls.auth_client = Client()
        cls.auth_client.force_login(cls.user)
        # Данные для POST-запроса при создании комментария.
        cls.form_data = {'text': cls.COMMENT_TEXT}

    def test_anonymous_user_cant_create_comment(self):
        # Совершаем запрос от анонимного клиента, в POST-запросе отправляем
        # предварительно подготовленные данные формы с текстом комментария.
        self.client.post(self.url, data=self.form_data)
        # Считаем количество комментариев.
        comments_count = Comment.objects.count()
        # Ожидаем, что комментариев в базе нет - сравниваем с нулём.
        self.assertEqual(comments_count, 0)

    def test_user_can_create_comment(self):
        # Совершаем запрос через авторизованный клиент.
        response = self.auth_client.post(self.url, data=self.form_data)
        # Проверяем, что редирект привёл к разделу с коментами.
        self.assertRedirects(response, f'{self.url}#comments')
        # Считаем количество комментариев.
        comments_count = Comment.objects.count()
        # Убеждаемся, что есть один комментарий.
        self.assertEqual(comments_count, 1)
        # Получаем объект комментария из базы.
        comment = Comment.objects.get()
        # Проверяем, что все атрибуты комментария совпадают с ожидаемыми.
        self.assertEqual(comment.text, self.COMMENT_TEXT)
        self.assertEqual(comment.news, self.news)
        self.assertEqual(comment.author, self.user)

    def test_user_cant_use_bad_words(self):
        # Формируем данные для отправки формы; текст включает

```

```

# первое слово из списка стоп-слов.
bad_words_data = {'text': f'Какой-то текст, {BAD_WORDS[0]}, еще текст'}
# Отправляем запрос через авторизованный клиент.
response = self.auth_client.post(self.url, data=bad_words_data)
form = response.context['form']
# Проверяем, есть ли в ответе ошибка формы.
self.assertFormError(
    form=form,
    field='text',
    errors=WARNING
)
# Дополнительно убедимся, что комментарий не был создан.
comments_count = Comment.objects.count()
self.assertEqual(comments_count, 0)

```

```

class TestCommentEditDelete(TestCase):

```

```

    # Тексты для комментариев не нужно дополнительно создавать
    # (в отличие от объектов в БД), им не нужны ссылки на self или cls,
    # поэтому их можно перечислить просто в атрибутах класса.
    COMMENT_TEXT = 'Текст комментария'
    NEW_COMMENT_TEXT = 'Обновлённый комментарий'

```

```

    @classmethod

```

```

    def setUpTestData(cls):

```

```

        # Создаём новость в БД.
        cls.news = News.objects.create(title='Заголовок', text='Текст')
        # Формируем адрес блока с комментариями, который понадобится для тестов.
        news_url = reverse('news:detail', args=(cls.news.id,)) # Адрес новости.
        cls.url_to_comments = news_url + '#comments' # Адрес блока с

```

комментариями.

```

        # Создаём пользователя - автора комментария.
        cls.author = User.objects.create(username='Автор комментария')
        # Создаём клиент для пользователя-автора.
        cls.author_client = Client()
        # "Логиним" пользователя в клиенте.
        cls.author_client.force_login(cls.author)
        # Делаем всё то же самое для пользователя-читателя.
        cls.reader = User.objects.create(username='Читатель')
        cls.reader_client = Client()
        cls.reader_client.force_login(cls.reader)
        # Создаём объект комментария.
        cls.comment = Comment.objects.create(
            news=cls.news,
            author=cls.author,
            text=cls.COMMENT_TEXT
        )
        # URL для редактирования комментария.
        cls.edit_url = reverse('news:edit', args=(cls.comment.id,))
        # URL для удаления комментария.
        cls.delete_url = reverse('news:delete', args=(cls.comment.id,))
        # Формируем данные для POST-запроса по обновлению комментария.
        cls.form_data = {'text': cls.NEW_COMMENT_TEXT}

```

```

def test_author_can_delete_comment(self):
    # От имени автора комментария отправляем DELETE-запрос на удаление.
    response = self.author_client.delete(self.delete_url)
    # Проверяем, что редирект привёл к разделу с комментариями.
    self.assertRedirects(response, self.url_to_comments)
    # Заодно проверим статус-коды ответов.
    self.assertEqual(response.status_code, HTTPStatus.FOUND)
    # Считаем количество комментариев в системе.
    comments_count = Comment.objects.count()
    # Ожидаем ноль комментариев в системе.
    self.assertEqual(comments_count, 0)

def test_user_cant_delete_comment_of_another_user(self):
    # Выполняем запрос на удаление от пользователя-читателя.
    response = self.reader_client.delete(self.delete_url)
    # Проверяем, что вернулась 404 ошибка.
    self.assertEqual(response.status_code, HTTPStatus.NOT_FOUND)
    # Убедимся, что комментарий по-прежнему на месте.
    comments_count = Comment.objects.count()
    self.assertEqual(comments_count, 1)

def test_author_can_edit_comment(self):
    # Выполняем запрос на редактирование от имени автора комментария.
    response = self.author_client.post(self.edit_url, data=self.form_data)
    # Проверяем, что сработал редирект.
    self.assertRedirects(response, self.url_to_comments)
    # Обновляем объект комментария.
    self.comment.refresh_from_db()
    # Проверяем, что текст комментария соответствует обновленному.
    self.assertEqual(self.comment.text, self.NEW_COMMENT_TEXT)

def test_user_cant_edit_comment_of_another_user(self):
    # Выполняем запрос на редактирование от имени другого пользователя.
    response = self.reader_client.post(self.edit_url, data=self.form_data)
    # Проверяем, что вернулась 404 ошибка.
    self.assertEqual(response.status_code, HTTPStatus.NOT_FOUND)
    # Обновляем объект комментария.
    self.comment.refresh_from_db()
    # Проверяем, что текст остался тем же, что и был.
    self.assertEqual(self.comment.text, self.COMMENT_TEXT)

```